



Scanning 332 files with 86 Code rules:

CODE RULES

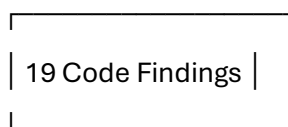
Language	Rules	Files	Origin	Rules
java	82	150	Community	86
kotlin	3	7		

SUPPLY CHAIN RULES

💎 Sign in with ``semgrep login`` and run ``semgrep ci`` to find dependency vulnerabilities and advanced cross-file findings.

PROGRESS

100% 0:00:00



masvs_crypto\src\main\java\com\dkronig\masvs_crypto\maswe_0009\EncryptionHandler.java

» java.lang.security.audit.crypto.desede-is-deprecated.desede-is-deprecated

Triple DES (3DES or DESede) is considered deprecated. AES is the recommended cipher.
Upgrade to use

AES.

Details: <https://sg.run/Geqn>

```
89 | KeyGenerator keyGenerator = KeyGenerator.getInstance(ALGORITHM_DES);
```

» java.lang.security.audit.crypto.des-is-deprecated.des-is-deprecated

DES is considered deprecated. AES is the recommended cipher. Upgrade to use AES. See
<https://www.nist.gov/news-events/news/2005/06/nist-withdraws-outdated-data-encryption-standard> for
more information.

Details: <https://sg.run/5Q73>

```
▶▶ | Autofix ▶ "AES/GCM/NoPadding"
```

```
89 | KeyGenerator keyGenerator = KeyGenerator.getInstance(ALGORITHM_DES);
```

```
: | -----
```

```
▶▶ | Autofix ▶ "AES/GCM/NoPadding"
```

```
118 | Cipher cipher = Cipher.getInstance(CIPHER_TRANSFORMATION);
```

```
: | -----
```

```
▶▶ | Autofix ▶ "AES/GCM/NoPadding"
```

```
133 | Cipher cipher = Cipher.getInstance(CIPHER_TRANSFORMATION);
```

masvs_crypto\src\main\java\com\dkronig\masvs_crypto\maswe_0010\EncryptionHandler.java

» java.lang.security.audit.cbc-padding-oracle.cbc-padding-oracle

Using CBC with PKCS5Padding is susceptible to padding oracle attacks. A malicious actor could

discern the difference between plaintext with valid or invalid padding. Further, CBC mode does not

include any integrity checks. Use 'AES/GCM/NoPadding' instead.

Details: <https://sg.run/ydxr>

▶▶ | Autofix ▶ "AES/GCM/NoPadding"

64 | Cipher cipher = Cipher.getInstance(CIPHER_TRANSFORMATION);

: | -----

▶▶ | Autofix ▶ "AES/GCM/NoPadding"

102 | Cipher cipher = Cipher.getInstance(CIPHER_TRANSFORMATION);

masvs_crypto\src\main\java\com\dkronig\masvs_crypto\maswe_0021\EncryptionHandler.java

» java.lang.security.audit.crypto.use-of-sha1.use-of-sha1

Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision resistant and is

therefore not suitable as a cryptographic signature. Instead, use PBKDF2 for password hashing or

SHA256 or SHA512 for other hash function applications.

Details: <https://sg.run/bXNp>

24 | MessageDigest digestAlgorithm =
MessageDigest.getInstance(HASHING_ALGORITHM);

masvs_crypto\src\main\java\com\dkronig\masvs_crypto\maswe_0022\EncryptionHandler.java

» java.lang.security.audit.crypto.no-static-initialization-vector.no-static-initialization-vector

Initialization Vectors (IVs) for block ciphers should be randomly generated each time they are used.

Using a static IV means the same plaintext encrypts to the same ciphertext every time, weakening the

strength of the encryption.

Details: <https://sg.run/BkB5>

17 | public class EncryptionHandler {

18 | private static final String SECRET_KEY_ALIAS = "maswe_0022_secret_key";

```

19 | private static final String ENCRYPTION_ALGORITHM = "AES";
20 | private static final String CIPHER_TRANSFORMATION = "AES/CBC/PKCS5PADDING";
21 | private static final String ENCRYPTION_KEY = "encryption_key";
22 | private static final int KEY_SIZE = 256;
23 |
24 | private static SharedPreferences sharedPreferences;
25 | private static byte[] iv = {47, -98, 3, 120, 14, -55, 89, 6, -12, 33, 9, -44, 63, -1,
    | 77, 22};
26 |
    | [hid 121 additional lines, adjust with --max-lines-per-finding]

```

masvs_platform\src\main\java\com\dkronig\masvs_platform\maswe_0053\EncryptionHandler.java

java.lang.security.audit.crypto.gcm-detection.gcm-detection

GCM detected, please check that IV/nonce is not reused, an Initialization Vector (IV) is a nonce

used to randomize the encryption, so that even if multiple messages with identical plaintext are

encrypted, the generated corresponding ciphertexts are different. Unlike the Key, the IV usually

does not need to be secret, rather it is important that it is random and unique. Certain encryption

schemes the IV is exchanged in public as part of the ciphertext. Reusing same Initialization Vector

with the same Key to encrypt multiple plaintext blocks allows an attacker to compare the ciphertexts

and then, with some assumptions on the content of the messages, to gain important information about

the data being encrypted.

Details: <https://sg.run/BLLb>

```

56 | Cipher cipher = Cipher.getInstance(AES_MODE);
    | :|-----

```

```
88 | Cipher cipher = Cipher.getInstance(AES_MODE);
```

```
: | -----
```

```
89 | GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
```

masvs_platform\src\main\java\com\dkronig\masvs_platform\maswe_0055\EncryptionHandler.java

› java.lang.security.audit.crypto.gcm-detection.gcm-detection

GCM detected, please check that IV/nonce is not reused, an Initialization Vector (IV) is a nonce

used to randomize the encryption, so that even if multiple messages with identical plaintext are

encrypted, the generated corresponding ciphertexts are different. Unlike the Key, the IV usually

does not need to be secret, rather it is important that it is random and unique. Certain encryption

schemes the IV is exchanged in public as part of the ciphertext. Reusing same Initialization Vector

with the same Key to encrypt multiple plaintext blocks allows an attacker to compare the ciphertexts

and then, with some assumptions on the content of the messages, to gain important information about

the data being encrypted.

Details: <https://sg.run/BLLb>

```
55 | Cipher cipher = Cipher.getInstance(AES_MODE);
```

```
: | -----
```

```
87 | Cipher cipher = Cipher.getInstance(AES_MODE);
```

```
: | -----
```

```
88 | GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
```

masvs_platform\src\main\java\com\dkronig\masvs_platform\maswe_0067\EncryptionHandler.java

› java.lang.security.audit.crypto.gcm-detection.gcm-detection

GCM detected, please check that IV/nonce is not reused, an Initialization Vector (IV) is a nonce

used to randomize the encryption, so that even if multiple messages with identical plaintext are

encrypted, the generated corresponding ciphertexts are different. Unlike the Key, the IV usually

does not need to be secret, rather it is important that it is random and unique. Certain encryption

schemes the IV is exchanged in public as part of the ciphertext. Reusing same Initialization Vector

with the same Key to encrypt multiple plaintext blocks allows an attacker to compare the ciphertexts

and then, with some assumptions on the content of the messages, to gain important information about

the data being encrypted.

Details: <https://sg.run/BLLb>

```
55 | Cipher cipher = Cipher.getInstance(AES_MODE);  
   | :|-----  
87 | Cipher cipher = Cipher.getInstance(AES_MODE);  
   | :|-----  
88 | GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);
```

masvs_storage\src\main\java\com\dkronig\masvs_storage\maswe_0006\EncryptionHandler.java
a

» java.lang.security.audit.crypto.des-is-deprecated.des-is-deprecated

DES is considered deprecated. AES is the recommended cipher. Upgrade to use AES. See

<https://www.nist.gov/news-events/news/2005/06/nist-withdraws-outdated-data-encryption-standard> for

more information.

Details: <https://sg.run/5Q73>

►► | Autofix ► "AES/GCM/NoPadding"

```
26 | Cipher cipher = Cipher.getInstance(ALGORITHM);
```

:|-----

▶▶| Autofix ▶ "AES/GCM/NoPadding"

42| Cipher cipher = Cipher.getInstance(ALGORITHM);

Scan Summary

✅ Scan completed successfully.

- Findings: 19 (19 blocking)
- Rules run: 82
- Targets scanned: 157
- Parsed lines: ~100.0%
- Scan skipped:
 - Files larger than files 1.0 MB: 1
 - Files matching .semgrepignore patterns: 27
- For a detailed list of skipped files and lines, run semgrep with the --verbose flag

Ran 82 rules on 157 files: 19 findings.

💎 Missed out on 29 pro rules since you aren't logged in!

⚡ Supercharge Semgrep OSS when you create a free account at <https://sg.run/rules>.